

Structure and Interpretation of Computer Programs (SICP)

Personal Study Notes

Miquéias Alves Medeiros

September 20, 2026

Notes based on the work of Harold Abelson, Gerald Jay Sussman, and Julie Sussman.

1 Notes on Ch. 1 Building Abstractions with Procedures — 1.1.3 Evaluating Combinations, pp. 12-15

To evaluate combinations, you need to evaluate the subexpressions first and then apply the procedure of the operator in the leftmost to the arguments that are the values of the other subexpressions (the operands).

As we've seen before, this expression could be, in mathematical notation, the following expression: $(3 \times 5) + (10 - 5)$.

```
1 (+ (* 3 5) (- 10 5))
```

To evaluate this expression as a whole, we need first evaluate the subexpressions $(* 3 5)$ and then $(- 10 5)$. So, $(* 3 5) \rightarrow (3 \times 5) = 15$, and $(- 10 5) \rightarrow (10 - 5) = 5$. Thus, our expression become: $(+ 15 5)$, which is $(15 + 5) = 20$.

```
1 (+ (* 3 5) (- 10 5)) ; 20
```

With this, we evaluate the entire expression by first breaking it down into small pieces and then combining to reach the final result. This is procedural thinking.

As you can see, the evaluation rule can be *recursive*, that is, its substeps can invoke the rule itself again, like evaluating subexpressions, where you need to apply the procedure of the operator in the leftmost, and then extend it to the operands. For example:

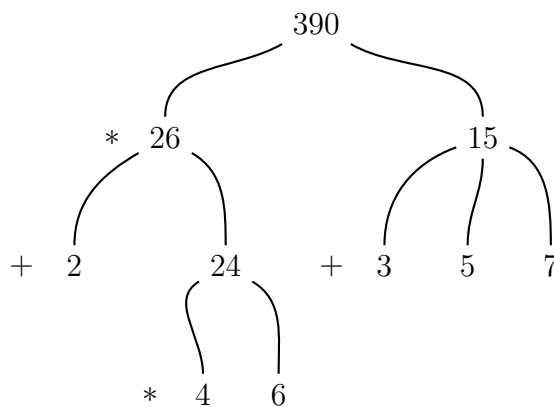


Figure 1.1: Tree representation, showing the value of each subcombination.

The expression $(* 26 15)$ is the full combination, whose value is 390. Each subcombination in parenthesis corresponds to a node of the tree, with branches corresponding to the operator and the operands of the combination resulting from it. If you analyse the tree bottom up, you'll realize it's the same as evaluating each subexpression and propagating the values outward, just like we did before, breaking it down into small pieces and reassembling them part by part.

In code, we get this:

```
1 (* (+ 2 (* 4 6)) ; subcombination -> 26
2 (+ 3 5 7)) ; subcombination -> 15
```

Evaluating from the inside out:

```
(* 4 6) => 24
(+ 2 24) => 26
(+ 3 5 7) => 15
(* 26 15) => 390
```

This kind of recursion is a very powerful technique for dealing with hierarchical tree-like objects. In fact, the “percolate values upward” form of the evaluation rule is an example of a general kind of process known as *tree accumulation*. Tree accumulation is a method of building or managing tree data structures, where nodes are added based on certain rule or condition. This process often involves recursive algorithms to traverse and manipulate the tree efficiently.

See: https://grokipedia.com/page/tree_accumulation

But how does interpreter essentially determine the value of a received expression? We have some cases that exemplifies this question.

1. The number itself. The values of the numerals are the numbers they represent.
2. The values of built-in operators. Operators such as “+” carry within themselves an instruction that the machine knows how to execute, which in this specific case is addition or sum.
3. The names in the environment that carry values associated with computational objects (variables). We’ve seen the concepts of naming and environment previously in the file: `C01-S1.1.2-P002_naming_and_environment.scm`

The second rule is a special case of the third one, because the symbols (operators) are also included in the environment, and their respective “values” are precisely the sequence of machine instructions they carry. The whole point here is the environment determines the meaning of these symbols and expressions. For example, to perform the operation `(+ x 1)` we need first to define in the environment who is “x”, because if this is not done, x has no value. So

```
1 (define x 10) ; Here, we're specifying to the environment that x has the value
    10.
```

Now we can perform the previously operation, because now x is recognized and associated by the environment. In other words, it now has meaning:

```
1 (+ x 1) ; 11
```

Note that the evaluation rule does not apply to definitions, because the role of **define** is simply associate a name with a value; therefore, it is not a combination. This kind of exceptions is called *special forms* and **define** is the only example of a special form that we’ve seen so far. Each special form has its own evaluation rule, keep that in mind.

References

- [1] ABELSON, Harold; SUSSMAN, Gerald Jay; SUSSMAN, Julie. *Structure and Interpretation of Computer Programs*. 2. ed. Cambridge, MA: MIT Press, 1996.